



C S C 3 3 6 0 0 · F I N A L D E M O N S T R A T I O N

TradeConnect

A two-sided marketplace for employers and tradespeople

Next.js 14

Flask

MySQL

15 Tables

12 Triggers

3NF

D E M O W A L K T H R O U G H

Today's Agenda

01

Introduction

Team, business idea, tech stack, profit model

02

Use Cases & User Journey

How employers, tradespeople, and juniors flow through the platform

03

MVP Demo 1 — Job Lifecycle

Posting → application → booking auto-creation

04

MVP Demo 2 — Endorsement

Junior unlocks job access through senior approval

05

Revenue Stream Reporting

Admin dashboard, fee %, totals & breakdown

06

Stored Procedures & Triggers

Two live trigger demos from MVP flows

07

2NF Relational Model

Final ERD with all entities and relationships

08

Takeaways

What we learned about relational databases

09

Extra Credit Work

Above-spec features we shipped

10

Q&A · Submission

Discussion and GitHub Classroom hand-in

Introduction

Team

1 **Oliver P Israel**
Backend

2 **Md Hasan**
Database

3 **Aung Latt**
Frontend

4 **Nathan Chen**
DevOps

5 **Dario Haxhia**
Reports

Business Idea

TradeConnect is a two-sided marketplace where employers post jobs and verified tradespeople apply, get booked, and get paid. Junior tradespeople earn credibility through senior endorsements.

Tech Stack

Next.js 14 (TypeScript) on the front, Flask + native mysql-connector on the back, MySQL 8 as the source of truth. No ORM — direct SQL with parametrized queries.

Profit Model

Two revenue streams: a 10% platform service fee on every paid booking, plus tiered tradesperson subscriptions (Free / Pro / Elite). Both flow into one revenue_streams table.

TWO USER JOURNEYS

How Value Flows Through the Platform



Employer journey — posting work and getting it done

1 Post a job

Title, trade type, city, budget, and scheduled date.



2 Review applicants

Browse tradespeople by rating, experience, proposed price.



3 Accept application

Trigger auto-creates the booking and closes the posting.



4 Pay & review

10% fee deducted server-side; trigger queues a review request.



Tradesperson / Junior journey — earning work and reputation

1 Register & list services

Set hourly rates and trade categories.



2 Apply / get endorsed

Juniors request a senior's endorsement to unlock jobs.



3 Get booked

City-match and time-overlap triggers guard the calendar.



4 Earn & subscribe

Pro / Elite plans lift the job cap; ratings update live.

LIVE DEMO · MVP #1

Job Lifecycle — From Posting to Booking

What we'll demo live

The end-to-end job pipeline — from an employer creating a posting through to a booking being auto-generated by a database trigger.

1

Employer logs in

Hits /employer/dashboard, opens 'New job posting'.

2

Submit a posting

POST /api/job-postings — row lands in job_postings (status='open').

3

Tradesperson applies

POST /api/job-applications with a proposed price and message.

4

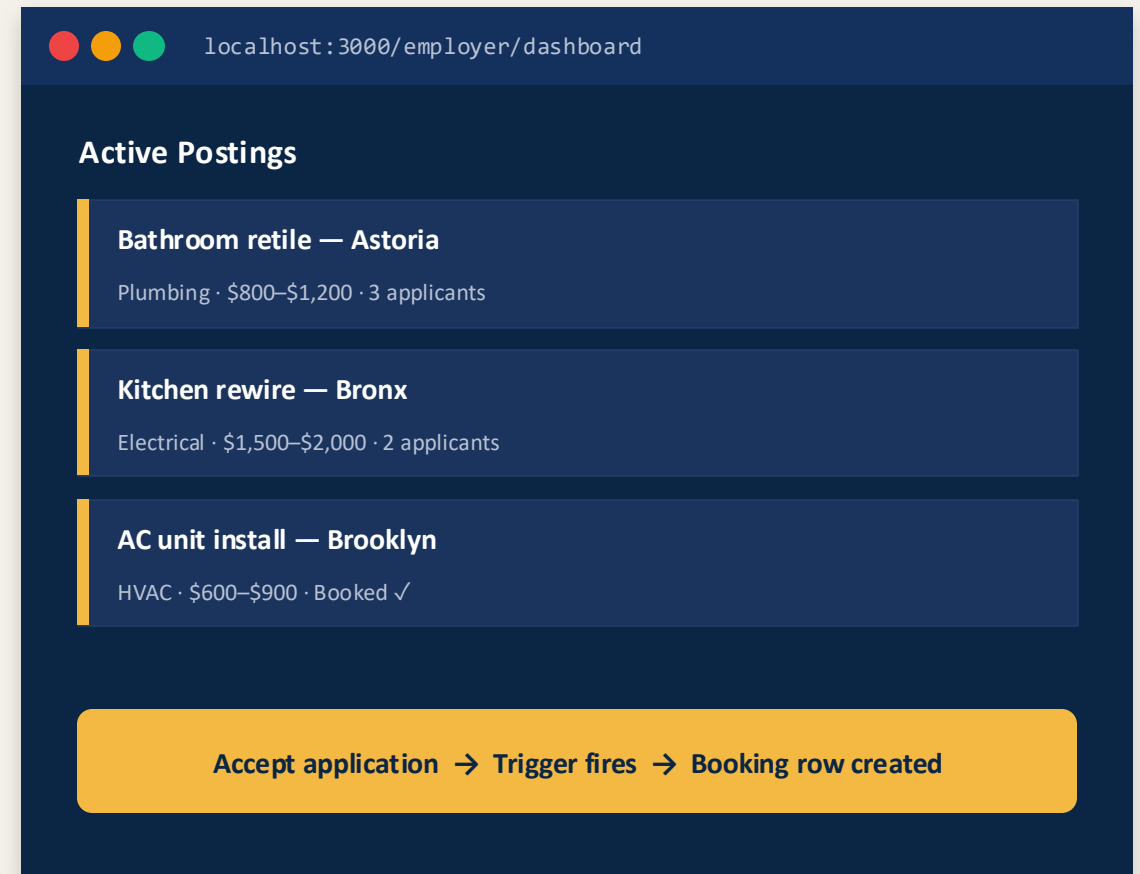
Employer accepts

PATCH on application status → 'accepted'.

5

Trigger fires

application_accept_creates_booking inserts the booking & closes the posting in one step.



localhost:3000/employer/dashboard

Active Postings

- Bathroom retile — Astoria**
Plumbing · \$800–\$1,200 · 3 applicants
- Kitchen rewire — Bronx**
Electrical · \$1,500–\$2,000 · 2 applicants
- AC unit install — Brooklyn**
HVAC · \$600–\$900 · Booked ✓

Accept application → Trigger fires → Booking row created

LIVE DEMO · MVP #2

Junior Endorsement — Unlocking Job Access

What we'll demo live

A new junior tradesperson requests endorsement from a verified senior. When the senior approves, a trigger writes the endorse_id back — instantly unlocking the junior's ability to take jobs.

1

Junior signs up

POST /api/auth/register with user_type='Junior'. endorse_id is NULL — they cannot book yet.

2

Upload proof

Multipart upload to /api/documents (path-traversal & MIME-spoof defenses applied).

3

Request endorsement

POST /api/endorsement-requests, status='pending'.

4

Senior approves

PATCH status → 'approved'.

5

Trigger fires

approve_endorsement_sets_endorse_id writes the senior's tradesperson_id to the junior's row — instant unlock.

triggers.sql · approve_endorsement_sets_endorse_id

```
CREATE TRIGGER approve_endorsement_sets_endorse_id
AFTER UPDATE ON endorsement_requests
FOR EACH ROW
BEGIN
  IF NEW.status = 'approved' AND OLD.status <> 'approved' THEN
    UPDATE tradespeople
      SET endorse_id = NEW.supervisor_tradesperson_id
      WHERE tradesperson_id = NEW.junior_tradesperson_id;
  END IF;
END;
```

Approval → endorse_id written → Junior can now be booked ✓



platform_settings		
setting_id	integer	
platform_fee_percentage	decimal(5,2)	NN
updated_at	timestamp	

tradespeople		
tradesperson_id	integer	
user_id	integer	NN
trade_category	varchar(100)	NN
license_number	varchar(100)	
license_state	varchar(50)	
license_expiry	date	
experience_year	integer	NN
endorse_id	integer	
job_limit	integer	
avg_rating	decimal(3,2)	
is_verified	boolean	

subscription_plans		
plan_id	integer	
plan_name	varchar(100)	NN
description	text	
monthly_price	decimal(10,2)	NN
job_limit	integer	NN
is_active	boolean	
created_at	timestamp	

revenue_streams		
revenue_stream_id	integer	
stream_name	varchar(100)	NN
description	text	
is_active	boolean	

users		
user_id	integer	
first_name	varchar(100)	NN
last_name	varchar(100)	NN
email	varchar(150)	NN
password_hash	varchar(255)	NN
phone	varchar(20)	
address	varchar(255)	
city	varchar(100)	
state	varchar(2)	
zip	varchar(20)	
user_type	varchar	NN
is_active	boolean	
last_login_at	timestamp	
created_at	timestamp	

services		
service_id	integer	
tradesperson_id	integer	NN
service_name	varchar(150)	NN
description	text	
hourly_rate	decimal(10,2)	
trade_type	varchar(100)	

endorsement_requests		
endorsement_request_id	integer	
junior_tradesperson_id	integer	NN
supervisor_tradesperson_id	integer	NN
message	text	
status	varchar	NN
decision_note	text	
created_at	timestamp	
decided_at	timestamp	
pending_marker	integer	

subscriptions		
subscription_id	integer	
tradesperson_id	integer	NN
plan_id	integer	NN
price_at_purchase	decimal(10,2)	NN
status	varchar	
start_date	date	NN
end_date	date	

job_postings		
job_posting_id	integer	
employer_id	integer	NN
title	varchar(200)	NN
description	text	
trade_type	varchar(100)	NN
city	varchar(100)	
address	varchar(255)	
budget_min	decimal(10,2)	
budget_max	decimal(10,2)	
scheduled_at	datetime	NN
status	varchar	NN
created_at	timestamp	
closed_at	timestamp	

documents		
document_id	integer	
uploaded_by_user_id	integer	NN
original_filename	varchar(255)	NN
storage_filename	varchar(255)	NN
mime_type	varchar(100)	NN
file_size_bytes	bigint	NN
related_entity_type	varchar	NN
related_entity_id	integer	
created_at	timestamp	

review_requests		
review_request_id	integer	
booking_id	integer	NN
employer_id	integer	NN
tradesperson_id	integer	NN
status	varchar	
created_at	timestamp	
submitted_at	timestamp	

reviews		
review_id	integer	
booking_id	integer	NN
reviewer_user_id	integer	NN
tradesperson_id	integer	NN
rating	integer	NN
comment	text	
created_at	timestamp	

bookings		
booking_id	integer	
user_id	integer	NN
tradesperson_id	integer	NN
service_id	integer	NN
application_id	integer	
scheduled_at	datetime	NN
status	varchar	NN
city	varchar(100)	
address	varchar(255)	
quoted_price	decimal(10,2)	
created_at	timestamp	

job_applications		
application_id	integer	
job_posting_id	integer	NN
tradesperson_id	integer	NN
service_id	integer	
proposed_price	decimal(10,2)	NN
message	text	
status	varchar	NN
created_at	timestamp	
decided_at	timestamp	

payments		
payment_id	integer	
booking_id	integer	NN
amount	decimal(10,2)	NN
method	varchar	NN
status	varchar	NN
paid_at	timestamp	

platform_fees		
platform_fee_id	integer	
payment_id	integer	NN
revenue_stream_id	integer	NN
fee_amount	decimal(10,2)	NN
fee_percentage	decimal(5,2)	NN
created_at	timestamp	

DEMONSTRATION · MIN. 2

Stored Procedures & Triggers

We chose triggers — the project requirement allows "stored procedures (including triggers)". Both fire automatically inside MVP flows and prevent invalid state at the DB layer, not just the app layer.

1 application_accept_creates_booking

AFTER UPDATE on job_applications

PURPOSE

When an employer accepts a tradesperson's application, automatically insert the matching bookings row and close the posting — in one atomic step.

MVP TIE-IN

Powers MVP #1 — the 'Accept' button can't forget to create the booking, because the DB does it.

```
IF NEW.status = 'accepted' AND OLD.status <> 'accepted' THEN
  INSERT INTO bookings (...)
  SELECT jp.employer_id, ja.tradesperson_id,
         ja.service_id, NEW.application_id,
         jp.scheduled_at, 'accepted'
  FROM job_postings jp JOIN ja ON ...;
UPDATE job_postings SET status='filled'
WHERE job_posting_id = NEW.job_posting_id;
END IF;
```

2 update_avg_rating_after_review

AFTER INSERT on reviews

PURPOSE

When an employer submits a star rating, recompute the affected tradesperson's avg_rating from their full review history and write it back.

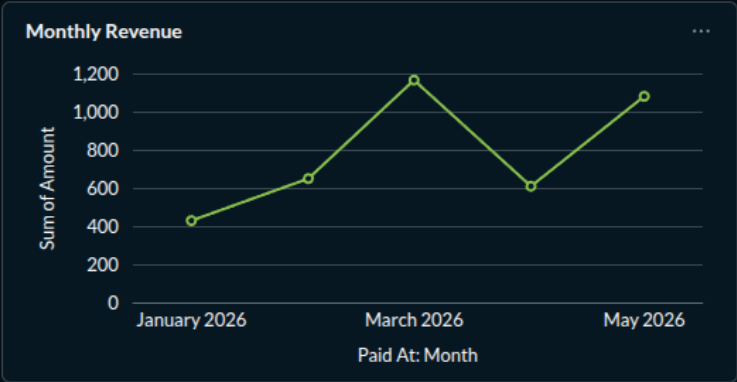
MVP TIE-IN

Powers the rating shown on every browse card and tradesperson profile — always fresh, always correct.

```
UPDATE tradespeople
  SET avg_rating = (
    SELECT ROUND(AVG(rating), 2)
    FROM reviews
    WHERE tradesperson_id = NEW.tradesperson_id
  )
WHERE tradesperson_id = NEW.tradesperson_id;
```

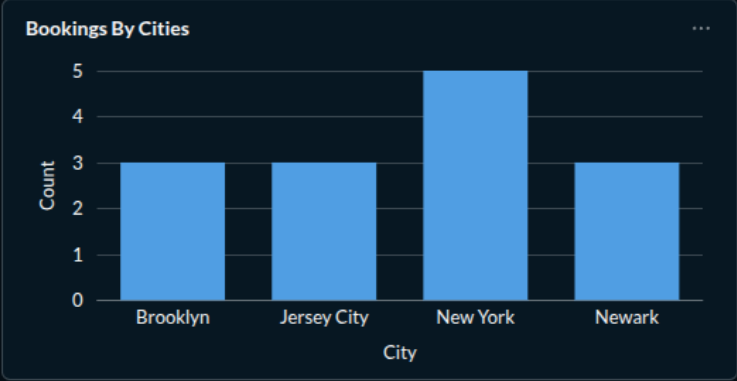
Analytics

🔗 📄 🕒 | 📌 ⓘ ⋮



Revenue

3,935



TradesPerson by Desc Rating

Tradesperson ID	User ID	Trade Category	License Number
1	2	Plumbing	
2	3	Electrical	
3	4	Plumbing	
4	6	Electrical	
5	11	Plumbing	
6	12	Electrical	
7	13	Carpentry	
8	14	Plumbing	

10 rows

WHAT WE LEARNED

Takeaways



NEW TECHNOLOGY

Triggers as a design tool

Before this project we thought of triggers as glorified audit logs. Building this app changed that. Once we moved 'accepting an application creates a booking' into a trigger, every code path that mutated applications automatically did the right thing — including a SQL console session. That's the kind of guarantee app-layer code can't give you.



AHA MOMENT

3NF isn't about purity — it's about avoiding silent bugs

Snapshotting subscription prices into price_at_purchase felt redundant until we realized: editing a plan price would have silently rewritten history for every past subscriber. Pulling the catalog out into subscription_plans and keeping a snapshot column was the single change that made the schema feel right.



WHAT WE'D TELL FUTURE STUDENTS

Push invariants down to the DB

City matching, time overlap, junior-without-endorsement — all of these started as if-statements in Flask. Every one of them ended up as a BEFORE INSERT/UPDATE trigger. The frontend got smaller, the API got simpler, and the rules can't be bypassed by a stray SQL UPDATE.

Extra Credit Work

Three categories of bonus work we did on top of the requirements.

01

UX-focused frontend

- 22 typed Next.js 14 routes, four role-specific dashboards.
- Reviews button auto-hides once a review is submitted (has_review flag joined server-side).
- Tailwind design system with consistent card / badge / form primitives.

02

Query optimization & data integrity

- 16 secondary indexes across the hottest join paths (bookings, applications, endorsements).
- Generated column + unique key blocks duplicate pending endorsement requests at the DB layer.
- All money math done with Python Decimal + ROUND_HALF_UP — never floats.

03

Dockerized deployment workflow

- Full app runs through Docker containers for consistent local setup.
- Docker separates frontend, backend, and database services for easier testing and handoff.
- Render deployment used for live hosting while Docker remains the main development/runtime environment.



O P E N F L O O R

Questions & Examination

We've reserved 5–8 minutes for questions from the audience. Ask anything about the schema, the triggers, the revenue logic, the UI, or our team workflow — we'll happily pop open the live app or the SQL console to show you.